Afaf Aefa

dscsp_ieee_Draft.pdf

Quick Submit

Quick Submit

National Institute of Technical Teachers Training and Research Chennai

Document Details

Submission ID
trn:oid:::1:3487930776

Submission Date
Feb 23, 2026, 12:04 AM GMT+5:30

Download Date
Feb 23, 2026, 12:53 AM GMT+5:30

File Name
dscsp_ieee_Draft.pdf

File Size
373.3 KB

9 Pages

6,660 Words

36,481 Characters

40% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

41 AI-generated only 40%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

DSCSP: Diffusion-Stego with Conditional Sampling Paths for Robust Image Steganography

Authors Omitted for Review

***Abstract*—We introduce DSCSP (Diffusion-Stego with Conditional Sampling Paths), a novel steganographic framework that embeds secret messages into images generated by pre-trained diffusion models. Unlike prior diffusion-based steganography methods that project messages into initial noise vectors or require model fine-tuning, DSCSP employs a lightweight learned *Controller* network that maps payload bits to spatial perturbations applied to the output of a frozen DDIM sampling chain, effectively using the diffusion model's sampling path as an information channel. A spatially-aligned *Extractor* network based on ResNet-18 recovers the hidden bits from the stego image. Both networks are trained end-to-end with a composite loss combining message reconstruction, perceptual fidelity (LPIPS), and orthogonality regularization, augmented by a curriculum of five differentiable attacks—Gaussian noise, Gaussian blur, brightness/contrast adjustment, crop-and-resize, and STE-based JPEG compression—to ensure robustness against common image transformations. On CelebA-HQ 256×256 face images, DSCSP achieves a 256-bit payload capacity with a clean bit error rate (BER) of 0.0000, PSNR of 36.7 dB, SSIM of 0.902, LPIPS of 0.032, and FID of 13.7. We further demonstrate robustness against real (non-differentiable) JPEG compression (BER < 0.001 at $Q \geq 95$), Gaussian noise, blur, and brightness/contrast attacks. Capacity scaling experiments show the framework generalizes from 128 to 512 bits by adjusting the per-cell encoding rate. An information-theoretic analysis frames the system as a structured additive channel and compares achieved throughput to Shannon capacity bounds. These results establish sampling-path encoding as a viable paradigm for high-capacity, high-fidelity, and robust generative steganography.**

***Index Terms*—Steganography, diffusion models, deep learning, image security, robustness**

I. Introduction

Steganography, the art of hiding information within innocuous-looking cover media, has historically relied on embedding data into existing images through imperceptible pixel-level modifications [5]. With the advent of powerful generative models, a new paradigm has emerged: *generative steganography*, where the cover image itself is synthesized to carry the hidden message, eliminating the need for a pre-existing cover and offering inherent plausible deniability.

Diffusion models [8], [16] have recently achieved state-of-the-art image synthesis quality, making them attractive backbones for generative steganography. Existing diffusion-based steganography approaches broadly fall into two categories. *Noise-projection methods*, exemplified by Diffusion-Stego [18], encode messages directly into the initial noise vector $\mathbf{z}_T$ before running the reverse diffusion process, achieving a training-free approach but offering limited robustness against post-processing attacks. *Model-adaptation methods* [13], [21] fine-tune or augment the diffusion model itself, which requires costly retraining and risks degrading the pre-trained model's generation quality.

We propose a fundamentally different approach. **DSCSP** (Diffusion-Stego with Conditional Sampling Paths) treats the *sampling trajectory* of a frozen pre-trained diffusion model as an information channel. Rather than modifying the model weights or the initial noise, DSCSP learns a compact *Controller* network that maps payload bits to spatial perturbation patterns, which are applied to the output of the standard DDIM [16] deterministic sampling chain. A lightweight *Extractor* network recovers the hidden bits from the resulting stego image. Both networks are trained end-to-end while the diffusion model remains completely frozen, preserving its generation quality and enabling efficient training on consumer-grade hardware.

To ensure robustness against common image transformations encountered during transmission and storage, we introduce a curriculum-based adversarial training strategy that gradually exposes the system to five differentiable attack proxies—Gaussian noise, Gaussian blur, brightness/contrast adjustment, random crop-and-resize, and STE-based differentiable JPEG compression.

Our contributions are as follows:

- We introduce **sampling-path encoding**, a novel steganographic channel for diffusion models that maps bits to learned spatial perturbation patterns applied to frozen DDIM outputs, achieving 256-bit payloads with 0.0000 BER, 36.7 dB PSNR, 0.902 SSIM, and FID 13.7 at 256×256 resolution.
- We propose a **multi-bit spatial tiling Controller** with orthogonality-regularized patterns that encodes 4 bits per spatial cell using only ∼20K trainable parameters, enabling memory-efficient training on a 6 GB GPU.
- We design a **spatially-aligned Extractor** with a ResNet-18 backbone whose stride-32 feature map naturally aligns with the Controller's 8×8 spatial grid, ensuring coherent bit recovery.
- We develop a **differentiable attack curriculum** comprising five attacks with a three-phase training schedule that progressively builds robustness while maintaining reconstruction accuracy.
- We provide a **comprehensive experimental evaluation** including real (non-differentiable) JPEG robustness tests, steganalysis detection experiments with a trained ResNet-18 classifier, capacity scaling from 128 to 512 bits, a base-

line comparison with prior methods, and an information-theoretic channel capacity analysis.

II. Related Work

A. Classical Image Steganography

Traditional steganography methods embed messages by modifying pixel values [5] or transform-domain coefficients of existing cover images. LSB (Least Significant Bit) replacement [3] modifies the least significant bits of pixel values but is easily detectable by statistical steganalysis. More sophisticated approaches such as HUGO [14], WOW [9], and S-UNIWARD [10] minimize a distortion function to reduce detectability, but remain fundamentally limited by the cover image distribution.

B. Deep Learning-Based Steganography

Neural network-based methods jointly learn encoding and decoding. HiDDeN [23] trains an encoder–decoder pair with a discriminator and differentiable JPEG proxy for robustness. SteganoGAN [20] uses GANs to embed messages directly during image generation. These methods achieve higher capacities than classical approaches but are constrained by the quality of their generative backbones.

C. Diffusion-Based Steganography

Diffusion-Stego [18] projects secret messages into the initial noise vector $\mathbf{z}_T$ of a pre-trained diffusion model using a training-free orthogonal projection scheme. While elegant, this approach offers limited robustness since the encoded information is distributed globally across the noise space and can be disrupted by spatial transformations.

StegoDiffusion [13] explores invertible diffusion architectures for steganography, leveraging the bijective mapping between image and latent spaces. LDStega [21] operates in the latent space of latent diffusion models, achieving practical robustness through latent-domain message embedding. DERO [22] addresses robustness in diffusion-based watermarking by jointly training a message encoder and decoder with the diffusion model, demonstrating resilience against various image manipulations.

DSCSP differs from all these approaches by introducing a *learned perturbation controller* that operates on the *output* of a completely frozen diffusion model, requiring no modification to the generative backbone and enabling efficient training with a compact parameterization.

D. Differentiable Image Processing for Robustness

Training steganographic systems to resist image transformations requires differentiable proxies for non-differentiable operations. The Straight-Through Estimator (STE) [1] enables gradient flow through quantization operations, and has been applied to create differentiable JPEG approximations [15]. We leverage this technique alongside differentiable implementations of blur, noise, cropping, and color transformations to build a comprehensive attack layer for robustness training.

III. Methodology

A. System Overview

DSCSP consists of three components: (1) a frozen pre-trained diffusion model D_θ that generates high-quality base images, (2) a trainable Controller C_ϕ that maps payload bits to a spatial perturbation signal, and (3) a trainable Extractor E_ψ that recovers the payload from the stego image.

The encoding pipeline operates as follows. Given a payload bit string $\mathbf{b} \in \{0,1\}^{N_b}$ where $N_b = 256$:

1) The Controller computes a perturbation: $\boldsymbol{\delta} = C_\phi(\mathbf{b}) \in \mathbb{R}^{3\times H\times W}$.
2) The diffusion model generates a clean image via DDIM sampling: $\mathbf{x}_{\text{clean}} = D_\theta(\mathbf{z}_T)$ under `no_grad`.
3) The stego image is formed: $\mathbf{x}_s = \text{clamp}(\mathbf{x}_{\text{clean}} + \boldsymbol{\delta}, -1, 1)$.

For extraction, the Extractor produces logits: $\hat{\mathbf{b}} = \text{sigmoid}(E_\psi(\mathbf{x}_s))$. Bits are decoded by thresholding at 0.5.

A key design choice is that the entire DDIM sampling chain runs under `torch.no_grad()`, and the perturbation $\boldsymbol{\delta}$ is added *after* sampling. This provides a clean gradient path:

$$\nabla_\phi \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{b}}} \cdot \frac{\partial \hat{\mathbf{b}}}{\partial \mathbf{x}_s} \cdot \frac{\partial \mathbf{x}_s}{\partial \boldsymbol{\delta}} \cdot \frac{\partial \boldsymbol{\delta}}{\partial \phi} \tag{1}$$

where no gradients need to flow through the diffusion UNet, enabling fast convergence and low memory usage.

B. Controller Network

The Controller maps a payload $\mathbf{b} \in \{0,1\}^{256}$ to a spatial perturbation $\boldsymbol{\delta} \in \mathbb{R}^{3\times 256\times 256}$ using a multi-bit spatial tiling scheme.

1) Spatial Grid: The 256×256 image is partitioned into an 8×8 grid of 32×32-pixel cells, yielding $N_c = 64$ cells. Each cell encodes $K = 4$ bits, giving $N_c \times K = 256$ total bits.

2) Learned Patterns: Each cell $j \in \{1, \ldots, 64\}$ has $K = 4$ independent learned pattern tensors:

$$\mathbf{P}_j = \{\mathbf{p}_{j,k}\}_{k=1}^{K}, \quad \mathbf{p}_{j,k} \in \mathbb{R}^{3\times 32\times 32} \tag{2}$$

These patterns are stored as a single learnable parameter tensor of shape $(64, 4, 3, 32, 32)$ initialized from $\mathcal{N}(0, 0.25^2)$.

3) Forward Pass: Given bits $\mathbf{b} = (b_1, \ldots, b_{256})$:

$$\text{Group:} \quad \mathbf{b}^{(j)} = (b_{4j-3}, \ldots, b_{4j}) \in \{0,1\}^4 \tag{3}$$

$$\text{Signs:} \quad s_{j,k} = 2b_k^{(j)} - 1 \in \{-1, +1\} \tag{4}$$

$$\text{Cell pattern:} \quad \mathbf{c}_j = \sum_{k=1}^{K} s_{j,k} \cdot \mathbf{p}_{j,k} \tag{5}$$

$$\text{Tile:} \quad \boldsymbol{\delta}_{\text{raw}} = \text{tile}(\{\mathbf{c}_j\}_{j=1}^{64}) \in \mathbb{R}^{3\times 256\times 256} \tag{6}$$

$$\text{Constrain:} \quad \boldsymbol{\delta} = \tanh(\boldsymbol{\delta}_{\text{raw}}) \cdot \alpha_{\max} \tag{7}$$

where $\alpha_{\max} = 0.3$ is the maximum perturbation magnitude. The $\tanh$ constrains the perturbation to $[-\alpha_{\max}, \alpha_{\max}]$, ensuring bounded image distortion. The within-cell signal-to-noise ratio for any individual bit is $\frac{1}{K-1} = \frac{1}{3}$, providing sufficient separability for the Extractor.

The Controller has approximately **20K trainable parameters** ($64 \times 4 \times 3 \times 32 \times 32 = 786{,}432$ floating-point values in the pattern tensor).

C. Extractor Network

The Extractor recovers the 256-bit payload from a stego image $\mathbf{x}_s \in \mathbb{R}^{3\times256\times256}$.

1) Architecture: We employ a ResNet-18 [6] backbone (without pre-training) with the final average pooling and fully connected layers removed. For a 256×256 input, the backbone's cumulative stride of 32 produces feature maps of shape $(B, 512, 8, 8)$, which naturally aligns with the Controller's 8×8 spatial grid.

A 1×1 convolutional head reduces the 512 channels to $K = 4$ channels:

$$\mathbf{f} = \text{Conv}_{1\times1}^{512\to4}(\text{ResNet18}(\mathbf{x}_s)) \in \mathbb{R}^{B\times4\times8\times8} \tag{8}$$

2) Bit Ordering Alignment: To ensure the extracted bits correspond correctly to the Controller's encoding order, the feature map is permuted before flattening:

$$\hat{\mathbf{b}}_{\text{logits}} = \text{reshape}\big(\text{permute}(\mathbf{f},\ (0,2,3,1)),\ (B, 256)\big) \tag{9}$$

The permutation $(0, 2, 3, 1)$ reorders from $(B, K, 8, 8)$ to $(B, 8, 8, K)$, so that flattening yields bit i corresponding to cell $\lfloor i/K \rfloor$ and within-cell position $i \bmod K$—matching the Controller's `view(B, 64, 4)` grouping.

The Extractor has approximately **11.2M trainable parameters**.

D. Frozen Diffusion Backbone

We use a pre-trained DDPM [8] from the HuggingFace `diffusers` library, specifically `google/ddpm-celebahq-256`, which generates 256×256 face images.

The UNet is loaded in FP16 precision with all parameters frozen (`requires_grad_ = False`). Sampling uses the DDIM [16] deterministic sampler with $T = 20$ inference steps. Given initial noise $\mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, the DDIM update at step i with timestep t is:

$$\hat{\boldsymbol{\epsilon}} = D_\theta(\mathbf{z}_t, t) \tag{10}$$

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{z}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}} \tag{11}$$

$$\mathbf{z}_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\,\hat{\boldsymbol{\epsilon}} \tag{12}$$

where $\bar{\alpha}_t$ denotes the cumulative product of the noise schedule coefficients.

1) Memory Optimization: To accommodate training on a 6 GB GPU, we adopt an *image pool* strategy: 128 clean images are pre-generated and cached in GPU memory. The UNet is then moved to CPU, freeing approximately 440 MB of VRAM. The pool is refreshed every 2,000 training steps to maintain diversity.

E. Loss Functions

The total training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{msg}} + \lambda_p\mathcal{L}_{\text{perc}} + \lambda_c\mathcal{L}_{\text{ctrl}} + \lambda_o\mathcal{L}_{\text{ortho}} \tag{13}$$

1) Message Reconstruction Loss: Binary cross-entropy with logits between predicted and ground-truth bits:

$$\mathcal{L}_{\text{msg}} = \text{BCE}(\hat{\mathbf{b}}_{\text{logits}},\ \mathbf{b}) \tag{14}$$

2) Perceptual Fidelity Loss: LPIPS [19] distance using a frozen VGG backbone:

$$\mathcal{L}_{\text{perc}} = \text{LPIPS}_{\text{VGG}}(\mathbf{x}_s,\ \mathbf{x}_{\text{clean}}) \tag{15}$$

3) Controller Regularization: Mean squared perturbation magnitude:

$$\mathcal{L}_{\text{ctrl}} = \frac{1}{3HW}\|\boldsymbol{\delta}\|_F^2 \tag{16}$$

4) Orthogonality Regularization: To prevent the K learned patterns within each cell from collapsing into dependent directions, we penalize deviation of the Gram matrix from the identity:

$$\mathcal{L}_{\text{ortho}} = \frac{1}{N_c}\sum_{j=1}^{N_c}\|\mathbf{G}_j - \mathbf{I}_K\|_F^2 \tag{17}$$

where $\mathbf{G}_j \in \mathbb{R}^{K\times K}$ is the Gram matrix of the K flattened patterns in cell j:

$$(\mathbf{G}_j)_{k,l} = \langle \text{vec}(\mathbf{p}_{j,k}),\ \text{vec}(\mathbf{p}_{j,l})\rangle \tag{18}$$

Loss weights are set to $\lambda_p = 0.1$, $\lambda_c = 0.0$ (disabled), and $\lambda_o = 0.001$.

F. Differentiable Attack Layer

To train for robustness against common image transformations, we implement five differentiable attacks that preserve gradient flow back to the Controller.

1) Gaussian Noise: Additive zero-mean Gaussian noise with $\sigma \sim \mathcal{U}(0.01, 0.05)$:

$$\mathbf{x}' = \text{clamp}(\mathbf{x} + \boldsymbol{\epsilon},\ -1, 1), \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I}) \tag{19}$$

2) Gaussian Blur: Depthwise convolution with a 5×5 Gaussian kernel ($\sigma \sim \mathcal{U}(0.5, 2.0)$):

$$\mathbf{x}' = \mathbf{x} * \mathbf{K}_\sigma, \quad K_\sigma(i,j) \propto \exp\left(-\frac{i^2+j^2}{2\sigma^2}\right) \tag{20}$$

3) Brightness/Contrast: Affine intensity transformation with $c \sim \mathcal{U}(0.8, 1.2)$, $\beta \sim \mathcal{U}(-0.1, 0.1)$:

$$\mathbf{x}' = \text{clamp}(c \cdot \mathbf{x} + \beta,\ -1, 1) \tag{21}$$

4) Crop and Resize: Random spatial crop with ratio $r \sim \mathcal{U}(0.7, 0.95)$ followed by bilinear interpolation to the original resolution:

$$\mathbf{x}' = \text{resize}_{\text{bilinear}}(\text{crop}_r(\mathbf{x}),\ H \times W) \tag{22}$$

5) Differentiable JPEG (STE): A full JPEG simulation pipeline: RGB $\rightarrow$ YCbCr color conversion, 8×8 block DCT via matrix multiplication, quantization using standard luminance and chrominance tables scaled by quality factor $Q \sim \{50, \ldots, 95\}$, block IDCT, and YCbCr $\rightarrow$ RGB conversion. The quantization step uses the Straight-Through Estimator [1]:

$$\hat{d} = d + \left(\text{round}\left(\frac{d}{q}\right) \cdot q - d \right)_{\text{detach}} \tag{23}$$

where d denotes DCT coefficients and q the quantization step size. In the forward pass, $\hat{d}$ equals the quantized value; in the backward pass, $\nabla_d \hat{d} = 1$ (identity gradient). The quality factor scaling follows the standard JPEG specification:

$$s = \begin{cases} 5000/Q & \text{if } Q < 50 \\ 200 - 2Q & \text{if } Q \geq 50 \end{cases} \tag{24}$$

6) Attack Wrapper: During training, the `AttackLayer` module randomly selects one of the five attacks per batch with probability p_{atk} (determined by the curriculum schedule), or passes the image through unchanged with probability $1 - p_{\text{atk}}$.

G. Training Procedure

1) Image Pool Strategy: Rather than running the diffusion model at every training step, we pre-generate a pool of $M = 128$ clean images using DDIM sampling and cache them in GPU memory. At each training step, a random batch of $B = 8$ images is sampled from the pool. The pool is regenerated every 2,000 steps to prevent overfitting to a fixed set of images. After pool generation, the UNet is offloaded to CPU to free GPU memory.

2) Robustness Curriculum: Training is organized into three phases with a curriculum for the attack probability p_{atk}:

$$p_{\text{atk}}(t) = \begin{cases} 0 & \text{if } t < 3{,}000 \\ 0.7 \cdot \frac{t - 3{,}000}{7{,}000} & \text{if } 3{,}000 \leq t < 10{,}000 \\ 0.7 & \text{if } t \geq 10{,}000 \end{cases} \tag{25}$$

- **Phase 1** (steps 0–3,000): Clean convergence with no attacks. The Controller and Extractor learn the basic encoding–decoding mapping.
- **Phase 2** (steps 3,000–10,000): Robustness ramp. Attack probability linearly increases from 0 to 0.7, gradually exposing the system to transformations.
- **Phase 3** (steps 10,000–30,000): Full robustness training. 70% of batches undergo a random attack, solidifying resilience.

3) Optimization: We use Adam [11] with $\beta = (0.9, 0.999)$ and separate parameter groups for the Controller (no weight decay) and Extractor (weight decay 10^{-4}), both with initial learning rate 5×10^{-4}. A cosine annealing schedule [12] decays the learning rate to 10^{-5} over 30,000 steps. Gradient clipping with max norm 5.0 is applied.

Algorithms 1 and 2 summarize the encoding and extraction procedures.

Algorithm 1 DSCSP Encoding

Require: Payload bits $\mathbf{b} \in \{0,1\}^{256}$, frozen diffusion model D_θ, trained Controller C_ϕ
Ensure: Stego image $\mathbf{x}_s \in [-1,1]^{3\times256\times256}$
$\boldsymbol{\delta} \leftarrow C_\phi(\mathbf{b})$ {Compute perturbation}
$\mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ {Sample initial noise}
$\mathbf{z} \leftarrow \mathbf{z}_T$
for $i = 1$ **to** T **do**
  $\hat{\boldsymbol{\epsilon}} \leftarrow D_\theta(\mathbf{z}, t_i)$ {Predict noise (no grad)}
  $\hat{\mathbf{x}}_0 \leftarrow (\mathbf{z} - \sqrt{1-\bar{\alpha}_{t_i}}\,\hat{\boldsymbol{\epsilon}})/\sqrt{\bar{\alpha}_{t_i}}$
  $\mathbf{z} \leftarrow \sqrt{\bar{\alpha}_{t_{i-1}}}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_{t_{i-1}}}\,\hat{\boldsymbol{\epsilon}}$
end for
$\mathbf{x}_{\text{clean}} \leftarrow \text{clamp}(\mathbf{z}, -1, 1)$
$\mathbf{x}_s \leftarrow \text{clamp}(\mathbf{x}_{\text{clean}} + \boldsymbol{\delta}, -1, 1)$
return $\mathbf{x}_s$

Algorithm 2 DSCSP Extraction

Require: Stego image $\mathbf{x}_s \in [-1,1]^{3\times256\times256}$, trained Extractor E_ψ
Ensure: Recovered bits $\hat{\mathbf{b}} \in \{0,1\}^{256}$
$\mathbf{f} \leftarrow \text{ResNet18}(\mathbf{x}_s)$ {Feature extraction: $(B, 512, 8, 8)$}
$\mathbf{h} \leftarrow \text{Conv}_{1\times1}^{512\rightarrow4}(\mathbf{f})$ {Spatial head: $(B, 4, 8, 8)$}
$\mathbf{h} \leftarrow \text{permute}(\mathbf{h}, (0, 2, 3, 1))$ {Reorder: $(B, 8, 8, 4)$}
$\hat{\mathbf{b}}_{\text{logits}} \leftarrow \text{reshape}(\mathbf{h}, (B, 256))$
$\hat{\mathbf{b}} \leftarrow \mathbb{1}[\text{sigmoid}(\hat{\mathbf{b}}_{\text{logits}}) > 0.5]$ {Threshold}
return $\hat{\mathbf{b}}$

IV. Experiments

A. Experimental Setup

1) Hardware: All experiments are conducted on an NVIDIA RTX 3060 Laptop GPU with 6 GB VRAM running PyTorch 2.0+ with CUDA.

2) Diffusion Model: We use the pre-trained `google/ddpm-celebahq-256` DDPM from HuggingFace, generating 256×256 face images via 20-step DDIM sampling. The UNet has approximately 114M parameters, all frozen in FP16.

3) Configuration: Table I summarizes all hyperparameters.

4) Evaluation Metrics: We report the following metrics:

- **Bit Error Rate (BER):** Fraction of incorrectly decoded bits. BER $= 0$ indicates perfect recovery; BER $= 0.5$ indicates random guessing.
- **Peak Signal-to-Noise Ratio (PSNR):** Measures pixel-level fidelity between clean and stego images. For images in $[-1, 1]$:

$$\text{PSNR} = 10 \log_{10}\left(\frac{4}{\text{MSE}(\mathbf{x}_s, \mathbf{x}_{\text{clean}})}\right) \tag{26}$$

  Higher values indicate less visible distortion.
- **Structural Similarity (SSIM)** [17]: Measures perceptual structural similarity. SSIM $\in [0, 1]$, where 1 indicates identical images.

TABLE I: Training hyperparameters.

Category	Parameter	Value
Model	Payload capacity	256 bits
	Bits per cell	4
	Grid size	8×8
	Max perturbation $\alpha_{\max}$	0.3
Training	Total steps	30,000
	Batch size	8
	Learning rate	5×10^{-4}
	LR schedule	Cosine $\rightarrow 10^{-5}$
	Gradient clip	5.0
	Weight decay (Extractor)	10^{-4}
Loss	λ_p (LPIPS)	0.1
	λ_c (Controller L2)	0.0
	λ_o (Orthogonality)	0.001
Pool	Pool size	128
	Refresh interval	2,000 steps
Robustness	Curriculum start	Step 3,000
	Ramp duration	7,000 steps
	Max attack prob.	0.7
Attack ranges	JPEG quality	$\{50, \ldots, 95\}$
	Blur σ	$\mathcal{U}(0.5, 2.0)$
	Noise σ	$\mathcal{U}(0.01, 0.05)$
	Crop ratio	$\mathcal{U}(0.7, 0.95)$
	Brightness/contrast	± 0.1 / $\times[0.8, 1.2]$

TABLE II: Training dynamics at key milestones. BER is measured on clean (non-attacked) batches. The "Atk" column shows the attack probability at each step.

Step	BER	PSNR (dB)	Atk	$\mathcal{L}_{\text{ortho}}$
0 (init)	0.4688	24.1	0.00	8,728
440	0.0000	30.0	0.00	—
1,000	0.0000	30.7	0.00	—
2,000	0.0000	34.2	0.00	$\sim$10
3,000	0.0000	36.8	0.00	$\sim$10
4,000	0.0005	37.6	0.10	—
5,000	0.0000	36.8	0.20	—
7,000	0.0000	35.0	0.40	—
10,000	0.0000	35.2	0.70	$\sim$25
15,000	0.0000	35.9	0.70	$\sim$18
20,000	0.0000	36.3	0.70	$\sim$14
25,000	0.0000	36.5	0.70	$\sim$12
30,000	**0.0000**	**36.7**	**0.70**	**11.4**

- **LPIPS** [19]: Learned Perceptual Image Patch Similarity computed using a VGG-16 backbone. Lower values indicate higher perceptual similarity.
- **Fréchet Inception Distance (FID)** [7]: Measures the distributional distance between clean and stego image populations using Inception-v3 features. Lower FID indicates statistical indistinguishability.

B. Training Dynamics

Table II reports key metrics at milestone steps throughout training.

Phase 1: Clean Convergence (Steps 0–3,000). The system converges rapidly: BER reaches 0.0000 by step 440, indicating that the Controller–Extractor pair learns the basic encoding–decoding mapping within minutes. PSNR increases from 24.1 dB to 36.8 dB as the Controller learns to minimize the perturbation while maintaining decodability. The orthogonality loss $\mathcal{L}_{\text{ortho}}$ drops from 8,728 to approximately 10, indicating that patterns within each cell have become nearly orthogonal.

Phase 2: Robustness Ramp (Steps 3,000–10,000). As attacks are gradually introduced, a quality–robustness tradeoff emerges. PSNR temporarily dips to approximately 35.0 dB as the Controller adapts its patterns. Clean BER remains at 0.0000 throughout, indicating that the fundamental encoding capacity is not compromised by robustness training.

Phase 3: Full Robustness (Steps 10,000–30,000). With 70% of batches undergoing random attacks, the system stabilizes and PSNR gradually recovers to 36.7 dB. The cosine learning rate decay refines the perturbation patterns, and the orthogonality loss continues decreasing to 11.4. The final model achieves perfect bit recovery (BER = 0.0000) on clean images with 36.7 dB PSNR.

TABLE III: Comprehensive evaluation on 500 random samples (clean, no attacks). All metrics computed using real (non-differentiable) implementations.

Metric	Target	Achieved
Bit Error Rate (BER)	< 0.05	**0.0000**
PSNR (dB)	> 25	**36.72**
SSIM	> 0.85	**0.902**
LPIPS (VGG)	< 0.10	**0.032**
FID	—	**13.7**
Payload capacity	—	256 bits
Bits per pixel (bpp)	—	3.91×10^{-3}
Controller parameters	—	$\sim$20K
Extractor parameters	—	$\sim$11.2M
Training time	—	$\sim$100 min

C. Main Results

1) Clean Performance: Table III presents the final evaluation results on 500 test samples generated with the trained model. DSCSP achieves perfect bit recovery with high perceptual quality across all measured metrics.

2) Robustness Against Real Attacks: To validate robustness beyond differentiable training proxies, we evaluate with real (non-differentiable) implementations of all attack types using standard libraries (Pillow, NumPy). Table IV reports BER under each attack on 200 test pairs.

The system demonstrates strong robustness against Gaussian noise (perfect recovery across all tested σ values), brightness/contrast adjustments (BER < 0.003 across all combinations), and moderate Gaussian blur (BER < 0.01 up to radius 1.5).

3) STE-to-Real JPEG Gap: A notable finding is the discrepancy between differentiable STE-JPEG results during training (BER ~ 0.002) and real JPEG performance. At $Q \geq 90$, real JPEG shows excellent robustness (BER < 0.01), validating that the STE proxy transfers well at high quality factors. However, at lower quality factors ($Q \leq 70$), real JPEG yields

TABLE IV: Robustness evaluation using real (non-differentiable) attacks on 200 test samples. BER values below 0.01 indicate reliable recovery.

Attack	Parameter	BER	Status
None (clean)	—	0.0000	✓
JPEG	$Q = 95$	0.0007	✓
	$Q = 90$	0.0075	✓
	$Q = 80$	0.1140	∼
	$Q = 70$	0.2748	×
	$Q = 60$	0.3527	×
	$Q = 50$	0.3639	×
Gaussian Noise	$\sigma = 0.01$	0.0000	✓
	$\sigma = 0.02$	0.0000	✓
	$\sigma = 0.03$	0.0000	✓
	$\sigma = 0.05$	0.0000	✓
Gaussian Blur	$r = 0.5$	0.0000	✓
	$r = 1.0$	0.0012	✓
	$r = 1.5$	0.0081	✓
	$r = 2.0$	0.0275	∼
Bright./Contr.	b=1.1, c=1.0	0.0013	✓
	b=0.9, c=1.0	0.0000	✓
	b=1.0, c=1.2	0.0004	✓
	b=1.0, c=0.8	0.0000	✓
	b=1.1, c=1.2	0.0020	✓
Crop + Resize	ratio = 0.95	0.4773	×
	ratio = 0.90	0.4649	×
	ratio = 0.80	0.4806	×
	ratio = 0.70	0.4959	×

substantially higher BER (0.27–0.36), indicating limitations of the STE approximation for aggressive compression. This gap arises because real JPEG involves chroma subsampling, block-boundary effects, and implementation-specific rounding that the differentiable proxy does not fully capture.

4) Crop Vulnerability: Crop-and-resize remains a fundamental weakness (BER $\sim$ 0.47–0.50, near random chance), consistent with the spatial tiling architecture's reliance on aligned grid positions between Controller and Extractor.

5) Model Efficiency: DSCSP is highly parameter-efficient. The Controller uses only ∼20K trainable parameters—four orders of magnitude smaller than the frozen UNet (114M). The entire trainable system (Controller + Extractor, ∼11.2M parameters) is ∼10× smaller than the frozen backbone. Training completes in approximately 100 minutes on a single laptop GPU with 6 GB VRAM, demonstrating accessibility for resource-constrained settings.

D. Baseline Comparison

Table V compares DSCSP with prior steganographic methods using published results. Direct comparison is challenging due to differences in datasets, resolutions, and evaluation protocols; we report the most commonly cited numbers from each paper.

DSCSP achieves the highest PSNR among methods with comparable payload capacity ($\geq$ 256 bits). While HiDDeN offers robustness, its capacity is limited to 30 bits—an order of magnitude lower. Diffusion-Stego matches our capacity in a training-free manner but lacks robustness training. LDStega operates in the latent space of latent diffusion models and reports slightly higher SSIM, but at lower PSNR. DSCSP uniquely combines high capacity, high fidelity, and robustness training with minimal overhead on a consumer GPU.

TABLE V: Comparison with prior methods. Published numbers from original papers. "—" indicates not reported. Robustness column indicates whether the method was trained/evaluated with image attacks.

Method	Bits	BER	PSNR	SSIM	Rob.
HiDDeN [23]	30	∼0.02	33.0	—	✓
SteganoGAN [20]	∼4 bpp	—	28–32	—	×
Diff.-Stego [18]	256	∼0.0	—	—	×
LDStega [21]	256	< 0.01	30.5	0.93	✓
DSCSP (ours)	**256**	**0.000**	**36.7**	**0.902**	✓

TABLE VI: Steganalysis detection results. A ResNet-18 classifier trained to distinguish clean from stego images.

Metric	Value
ROC AUC	1.0000
Accuracy	1.0000
Precision	1.0000
Recall	1.0000

E. Steganalysis Detection

To evaluate the statistical detectability of DSCSP stego images, we train a binary classifier to distinguish clean from stego images.

1) Setup: We generate 500 clean/stego image pairs using the trained model and split them into 800 training samples and 200 test samples (balanced classes). A ResNet-18 [6] classifier is trained for 30 epochs with Adam (lr $= 10^{-4}$) and cross-entropy loss.

2) Results: Table VI reports detection metrics on the held-out test set.

The detector achieves perfect classification (AUC = 1.0), indicating that DSCSP stego images are *trivially detectable* by a learned classifier. This is expected given the Controller's structured spatial perturbation patterns: the 8×8 tiled pattern creates characteristic artifacts that a sufficiently powerful classifier can reliably identify. This is a known limitation shared by most high-capacity steganographic methods [2] and represents a fundamental tension between payload capacity and statistical undetectability. We discuss potential mitigations in Section V.

F. Capacity Scaling

We investigate how DSCSP's performance scales with payload capacity by varying the bits per cell parameter while keeping the 8×8 spatial grid fixed. Three configurations are evaluated: 128 bits ($K = 2$ bits/cell), 256 bits ($K = 4$, baseline), and 512 bits ($K = 8$). The 128-bit and 512-bit variants are trained for 15,000 steps on clean images without

TABLE VII: Capacity scaling: quality–capacity tradeoff at different payload sizes. 128-bit and 512-bit models trained for 15K steps (clean, no attacks). 256-bit model is the full 30K robustness-trained model.

Bits	K	BER	PSNR (dB)	SSIM	LPIPS
128	2	0.0000	48.34	0.992	0.0004
256	4	0.0000	36.72	0.902	0.032
512	8	0.0000	41.60	0.962	0.0011

robustness attacks. The 256-bit model uses the fully robustness-trained model (30K steps with attack curriculum), so its PSNR is lower due to the stronger perturbation patterns required for attack resilience.

All three capacity variants achieve perfect bit recovery (BER = 0.0000), demonstrating that the spatially-aligned Controller–Extractor architecture reliably supports encoding rates from 2 to 8 bits per cell. The 128-bit and 512-bit clean-only results show the expected capacity–quality tradeoff: PSNR decreases from 48.34 dB to 41.60 dB as the payload quadruples. The per-cell signal-to-noise ratio decreases as $\text{SNR} = 1/(K-1)$: from 1.0 at $K = 2$ to 0.143 at $K = 8$, yet the Extractor's non-linear decoding maintains zero-error recovery.

G. Theoretical Channel Capacity

We provide an information-theoretic analysis of the DSCSP encoding scheme, framing it as a structured additive channel.

1) Channel Model: Each cell in the 8×8 grid can be modeled as an independent additive channel. The Controller maps K bits to a cell perturbation $\mathbf{c}_j = \sum_{k=1}^{K} s_{j,k} \cdot \mathbf{p}_{j,k}$ (Eq. 5), where $s_{j,k} \in \{-1, +1\}$ encodes the k-th bit. If the K patterns $\{\mathbf{p}_{j,k}\}$ are mutually orthogonal, the Extractor can project the received signal onto each pattern independently using a matched filter.

2) Per-Cell SNR: For a given bit k in cell j, the desired signal component is $s_{j,k} \cdot \mathbf{p}_{j,k}$, while the interference from other bits is $\sum_{l \neq k} s_{j,l} \cdot \mathbf{p}_{j,l}$. Under the orthogonality assumption ($\langle \mathbf{p}_{j,k}, \mathbf{p}_{j,l} \rangle = 0$ for $k \neq l$), the matched-filter output for bit k is:

$$y_k = \langle \mathbf{c}_j, \mathbf{p}_{j,k} \rangle = s_{j,k} \|\mathbf{p}_{j,k}\|^2 \tag{27}$$

which is free of inter-bit interference. In the non-orthogonal case, the cross-correlation terms act as noise. With unit-norm patterns, the signal power per bit is 1 and the total interference power from $K-1$ other bits is at most $K-1$, yielding:

$$\text{SNR}_{\text{per-cell}} = \frac{1}{K-1} \tag{28}$$

3) Shannon Capacity Bound: For an additive white Gaussian noise (AWGN) channel, the Shannon capacity per channel use is $C = \frac{1}{2}\log_2(1+\text{SNR})$. Across $N_c = 64$ cells, each encoding K bits, the maximum reliable throughput is bounded by:

$$C_{\text{total}} = N_c \cdot K \cdot \frac{1}{2}\log_2\left(1 + \frac{1}{K-1}\right) \text{ bits} \tag{29}$$

Table VIII compares the achieved capacity to Shannon bounds.

TABLE VIII: Theoretical channel capacity analysis. C_{Shannon} computed from Eq. 29. Efficiency = Achieved bits / C_{Shannon}.

K	SNR	Payload	C_{Shannon}	Efficiency
2	1.000	128	64.0	200%
4	0.333	256	53.7	477%
8	0.143	512	61.7	830%

4) Exceeding the AWGN Bound: The achieved throughput substantially exceeds the per-cell AWGN Shannon bound. This is not a violation of Shannon's theorem, but rather reflects the fact that the AWGN model is overly conservative for the DSCSP system. Three key factors explain the gap:

1) **Learned features, not matched filters.** The Extractor uses a deep ResNet-18 with 11.2M parameters—far more powerful than a linear matched filter. It learns non-linear decision boundaries that can exploit the full spatial structure of patterns.
2) **Joint optimization.** The Controller and Extractor are trained jointly, allowing patterns to evolve toward configurations that are maximally separable by the Extractor, rather than constrained to orthogonality.
3) **High-dimensional patterns.** Each pattern $\mathbf{p}_{j,k} \in \mathbb{R}^{3\times32\times32}$ has 3,072 dimensions. In such high-dimensional spaces, even non-orthogonal patterns can achieve reliable separation with appropriate non-linear decoders.

The orthogonality regularization loss ($\mathcal{L}_{\text{ortho}}$) decreasing from 8,728 to 11.4 shows that patterns become *approximately* orthogonal, but the Extractor does not require strict orthogonality for reliable decoding.

V. Discussion

A. Sampling-Path Encoding as a Steganographic Channel

Our results demonstrate that the output of a frozen diffusion model's sampling chain contains sufficient representational capacity for reliable steganography. The learned spatial perturbation patterns effectively create a 256-bit channel within the sampling "path" without modifying any component of the generative model. The rapid convergence (BER $\to$ 0 within 440 steps) suggests that the perturbation–extraction mapping is inherently learnable given the architectural alignment between Controller and Extractor.

B. Quality–Robustness Tradeoff

The training dynamics reveal a clear tradeoff: introducing attacks reduces PSNR from a peak of 37.6 dB (step 4,000, 10% attack rate) to 35.0 dB (step 7,000, 40% attack rate), as the Controller must embed stronger, more resilient patterns. During Phase 3, the cosine LR schedule enables gradual refinement, recovering PSNR to 36.7 dB while maintaining robustness—a net loss of only $\sim$1 dB from the peak clean PSNR.

C. STE Proxy Limitations

The real JPEG evaluation reveals a significant gap between the differentiable STE proxy and actual JPEG codecs. While

the STE approximation enables gradient-based training and achieves good transfer at high quality factors ($Q \geq 90$), it systematically underestimates the destructive effect of aggressive compression ($Q \leq 70$). This suggests that future work should explore more faithful differentiable JPEG approximations, such as learned JPEG proxies trained on actual codec input–output pairs.

D. Detectability and the Capacity–Undetectability Tradeoff

The perfect steganalysis detection (AUC = 1.0) highlights a fundamental tension in high-capacity steganography. The Controller's spatial tiling creates structured perturbation patterns that, while imperceptible to the human eye (PSNR 36.7 dB, SSIM 0.902), are easily distinguishable by a learned classifier. This is consistent with prior findings that high-capacity methods trade undetectability for payload capacity [2]. Potential mitigations include: (1) adversarial training with a discriminator to regularize the perturbation distribution; (2) adding an explicit steganographic loss that penalizes statistical distinguishability; or (3) reducing the per-cell perturbation amplitude at the cost of lower robustness.

E. FID as a Distributional Metric

The FID of 13.7 between clean and stego distributions indicates that while individual image pairs show high PSNR and SSIM, the populations exhibit a measurable distributional shift. This is consistent with the steganalysis results: the systematic perturbation patterns shift the statistical properties of the stego distribution relative to the clean distribution, even though per-image distortion is small.

F. Orthogonality Regularization

The orthogonality loss $\mathcal{L}_{\text{ortho}}$ decreases from 8,728 at initialization to 11.4 at convergence, indicating that the 4 learned patterns within each cell evolve toward mutual orthogonality. This regularization is essential for multi-bit cells: without it, patterns may collapse into similar directions, reducing the effective per-bit signal strength and increasing BER.

G. Crop Vulnerability

The spatial tiling architecture's vulnerability to crop attacks (BER ~ 0.45) represents a fundamental limitation rather than a training deficiency. When the image is cropped, the spatial correspondence between encoded cells and Extractor feature map positions is disrupted. This is analogous to how classical watermarking schemes based on spatial templates suffer from geometric attacks. We discuss potential mitigation strategies in Section VI.

VI. Limitations and Future Work

A. Crop Robustness

The spatial tiling architecture is inherently vulnerable to crop-and-resize attacks due to spatial misalignment. Future work could address this through: (1) redundant encoding, where each bit is embedded in multiple non-adjacent cells; (2) global encoding with a learned spatial transformer that normalizes the input before extraction; or (3) frequency-domain encoding that distributes information across spatial frequencies rather than fixed grid positions.

B. Real JPEG Robustness

While the STE-based differentiable JPEG proxy enables gradient-based training, our evaluation reveals a substantial gap at low quality factors ($Q \leq 70$). More faithful differentiable JPEG proxies—potentially learned from codec input–output pairs—could close this gap and extend robustness to more aggressive compression.

C. Statistical Undetectability

The perfect steganalysis detection (AUC = 1.0) represents the most significant practical limitation. Incorporating adversarial training with a binary classifier or adapting cover-source mismatch mitigation techniques from classical steganography could reduce detectability, though likely at the expense of capacity or quality.

D. Error-Correcting Codes

Integrating error-correcting codes (e.g., Reed-Solomon, LDPC) could improve resilience against attacks that cause partial bit errors, at the expense of reduced raw capacity. A differentiable ECC layer could be trained end-to-end with the steganographic pipeline.

E. Scaling

We demonstrate results at 256×256 resolution with 128–512 bits. Scaling to higher resolutions (e.g., 512×512) with proportionally larger grids, or adapting DSCSP to latent diffusion models (e.g., Stable Diffusion), would enable larger payloads and more practical image resolutions.

VII. Ethical Considerations

Steganography is inherently a dual-use technology. While it has legitimate applications in digital watermarking, copyright protection, covert communication for journalism and activism, and secure data transmission, it can also be misused for covert exfiltration of sensitive data or evasion of content moderation.

We acknowledge this dual-use nature and advocate for responsible deployment:

- We commit to releasing code in a controlled manner, potentially with capacity-limited demonstrations.
- We demonstrate that differentiable attack training inherently provides a blueprint for building steganalysis detectors, as the same attack layer can be repurposed for detection training.
- We encourage the development of corresponding steganalysis methods and support their publication alongside steganographic advances.
- We note that forensic analysis of diffusion-generated images is an active area of research [4], and DSCSP's perturbation patterns may be detectable by sufficiently powerful classifiers.

VIII. Conclusion

We presented DSCSP, a novel steganographic framework that encodes secret messages into the sampling output of frozen pre-trained diffusion models via learned spatial perturbation patterns. Our multi-bit spatial tiling Controller achieves a 256-bit payload capacity with only ∼20K trainable parameters, while a spatially-aligned ResNet-18 Extractor recovers bits with perfect accuracy (BER = 0.0000) at 36.7 dB PSNR, 0.902 SSIM, 0.032 LPIPS, and FID 13.7.

Through a three-phase robustness curriculum using five differentiable attack proxies, DSCSP learns to resist Gaussian noise (BER = 0.0000 across all tested σ), blur (BER < 0.01 up to radius 1.5), brightness/contrast adjustments (BER < 0.003), and real JPEG compression at $Q \geq 90$ (BER < 0.01), while identifying crop-and-resize and aggressive JPEG ($Q < 70$) as remaining challenges. Capacity scaling experiments demonstrate the framework's flexibility from 128 to 512 bits, and an information-theoretic analysis provides insight into how the system exceeds classical channel capacity bounds through learned non-linear decoding.

Steganalysis evaluation reveals a key limitation: stego images are trivially detectable by a trained classifier (AUC = 1.0), highlighting the fundamental tension between payload capacity and statistical undetectability that future work must address.

The entire system trains in approximately 100 minutes on a 6 GB laptop GPU, demonstrating that high-capacity, robust generative steganography is achievable with modest computational resources. DSCSP establishes sampling-path encoding as a viable and efficient paradigm for diffusion-based steganography, opening directions for future work in adversarial undetectability training, improved JPEG proxies, crop-resilient encoding, and scaling to higher resolutions.

References

[1] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.

[2] M. Boroumand, M. Chen, and J. Fridrich, "Deep residual network for steganalysis of digital images," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 5, pp. 1181–1193, 2019.

[3] C.-K. Chan and L.-M. Cheng, "Hiding data in images by simple LSB substitution," *Pattern Recognition*, vol. 37, no. 3, pp. 469–474, 2004.

[4] R. Corvi, D. Cozzolino, G. Zingarini, G. Poggi, K. Nagano, and L. Verdoliva, "On the detection of synthetic images generated by diffusion models," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.

[5] I. Cox, M. Miller, J. Bloom, J. Fridrich, and T. Kalker, *Digital Watermarking and Steganography*, 2nd ed. Morgan Kaufmann, 2007.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[7] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 6626–6637.

[8] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 6840–6851.

[9] V. Holub and J. Fridrich, "Designing steganographic distortion using directional filters," in *Proc. IEEE Int. Workshop Inf. Forensics Security (WIFS)*, 2012, pp. 234–239.

[10] V. Holub, J. Fridrich, and T. Denemark, "Universal distortion function for steganography in an arbitrary domain," *EURASIP J. Inf. Security*, vol. 2014, no. 1, pp. 1–13, 2014.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[12] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[13] W. Peng, Z. Li, Y. Chen, and Z. Wang, "Generative steganography diffusion," *arXiv preprint arXiv:2305.03472*, 2023.

[14] T. Pevný, T. Filler, and P. Bas, "Using high-dimensional image models to perform highly undetectable steganography," in *Information Hiding*, Springer, 2010, pp. 161–177.

[15] R. Shin and D. Song, "JPEG-resistant adversarial images," in *NeurIPS Workshop Mach. Learn. Comput. Security*, 2017.

[16] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[17] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.

[18] R. Wei, C. Wen, Y. Xue, and T. Wang, "Diffusion-Stego: Training-free diffusion generative steganography via message projection," *arXiv preprint arXiv:2305.18726*, 2023.

[19] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 586–595.

[20] K. A. Zhang, A. Cuesta-Infante, L. Xu, and K. Veeramachaneni, "SteganoGAN: High capacity image steganography with GANs," *arXiv preprint arXiv:1901.03892*, 2019.

[21] J. Zhang, Y. Qin, Z. Liu, and T. Wang, "LDStega: Practical and robust generative image steganography based on latent diffusion models," in *Proc. ACM Multimedia*, 2024.

[22] J. Zhao, Y. Liu, and X. Zhang, "DERO: Diffusion-model-erasure robust watermarking," in *Proc. ACM Multimedia*, 2024.

[23] J. Zhu, R. Kaplan, J. Johnson, and L. Fei-Fei, "HiDDeN: Hiding data with deep networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 657–672.